

2 Data Types and Data Structures

Learning Outcomes

After studying this chapter, students will be able to:

- Define data, data types, and data structures.
- Differentiate between atomic and composite data.
- Explain primitive and non-primitive data structures.
- Understand the concept of Abstract Data Types (ADT).
- Classify data structures as linear and non-linear.
- Explain the relationship between data types, data structures, and algorithms.

2.1 Introduction

Every computer program works with *data*. Before data can be processed, it must be represented in a suitable form. A programming language uses *data types* to define the kind of values that can be stored and the operations that can be performed on them. These data values are then organized using *data structures*, which enable efficient storage, retrieval, and manipulation.

Data types and data structures form the foundation of programming because every algorithm operates on specific data organized in an appropriate structure.

Understanding the fundamental concepts of data structures is essential before learning specific data structures and algorithms. The terms **data**, **data type**, **data object**, **data structure**, **data representation**, and **abstract data type (ADT)** describe different aspects of how information is stored, organized, and manipulated in a computer system. Although these terms are closely related, each has a distinct meaning.

2.2 Data

2.2.1 Definition

Data is a collection of raw facts, figures, symbols, or observations that can be processed to produce meaningful information. Data may consist of numbers, characters, images, audio, or any other form of digital content. By itself, data has little meaning until it is processed and interpreted.

2.2.2 Examples

Student ID: 1025

Student Name: Priya

Age: 19

Temperature: 36.8°C

Grade: A

2.2.3 Characteristics of Data

- Raw and unprocessed.
- Can be stored, retrieved, and manipulated.
- May be numeric, textual, graphical, audio, or video.
- Forms the basis for information processing.

2.3 Data Type

A **data type** specifies the set of values that can be stored and the operations that can be performed on those values. In simple terms, a data type tells the compiler what kind of data is stored and how it can be manipulated.

2.3.1 Definition

A data type is defined by a collection of values and a collection of valid operations. Every programming language provides a set of built-in data types, and many languages also support user-defined data types.

For example,

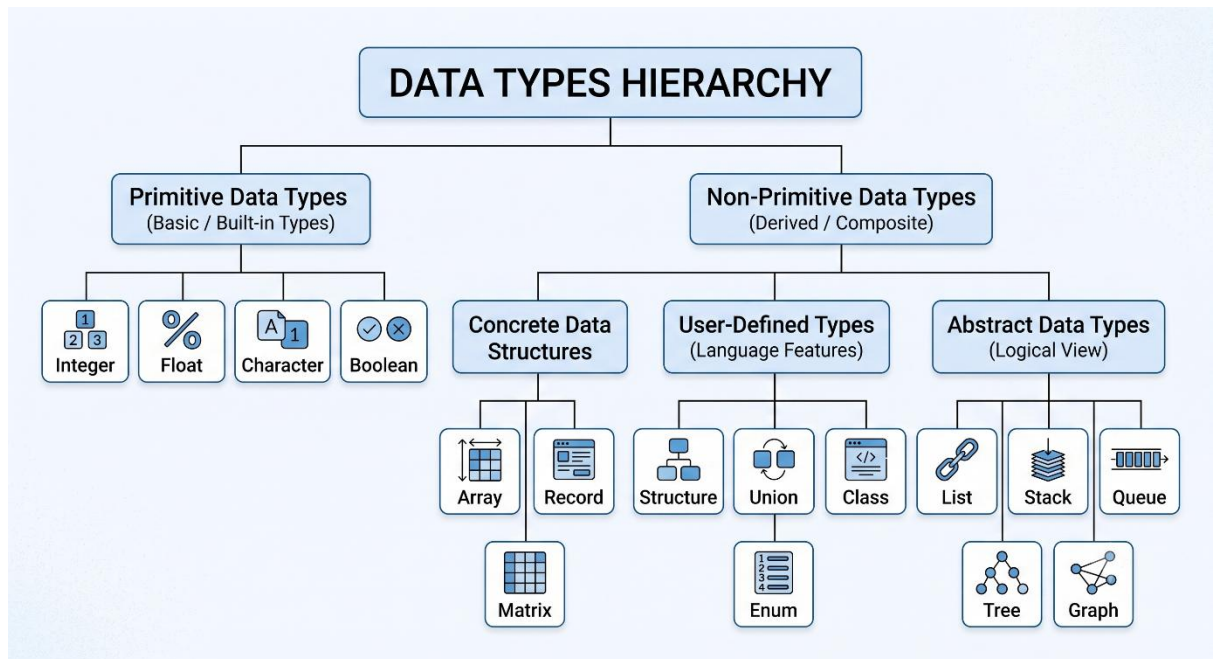
Integer

- values: ..., -2, -1, 0, 1, 2 ...
- Operations: +, -, ×, ÷, %, comparison

String

- values - represent sequences of characters
- operations - concatenation and comparison

2.3.2 Classification of Data Types



2.3.2.1 Primitive (Basic) Data Types / Primitive Data Structure

Primitive data types are the fundamental building blocks provided by a programming language. They store a single value and cannot be divided into smaller data types. These data types are not made of other data structures.

i. Characteristics

- Built into the programming language.
- Fixed memory size.
- Fast execution.
- Directly supported by hardware.

ii. Common Primitive Data Types

Data Type	Description	Example
Integer	Stores whole numbers	25
Character	Stores a single character	'A'
Boolean	Stores logical values	True
Float	Stores single-precision decimal values	12.75
Double	Stores double-precision decimal values	25.6789

iii. Memory Representation of Primitive Data type

Memory representation refers to how primitive data types are stored in the computer's main memory (RAM). Every primitive data type occupies a fixed amount of memory, and the value is represented internally using binary digits (0s and 1s). The memory required depends on the data type and the programming language. A computer stores all information in the form of binary digits. Memory is organized as a sequence of bytes, each identified by a unique memory address.

1 Bit = Smallest unit of memory (0 or 1)

8 Bits = 1 Byte

Bits	0	1	0	0	
Address	1001	1002	1003	1004	

Data Type	Size	Representation	Example	Memory (Bytes)
Integer	4 Bytes (32 bits)	Binary (Two's Complement)	25	Byte 1: 00000000 Byte 2: 00000000 Byte 3: 00000000 Byte 4: 00011001
Char	1 Byte (8 bits)	ASCII / Unicode	'A'	Byte 1: 01000001 (ASCII = 65)
Boolean	1 Byte	Binary (0=False, 1=True)	True	1 Byte: Byte 1: 00000001 (False = 00000000, True = 00000001)
Float	4 Bytes (32 bits)	IEEE 754 Single Precision (1 sign bit, 8 exponent bits, 23 fraction/mantissa bits)	12.75	Binary: Byte 1: 01000001 Byte 2: 01001100 Byte 3: 00000000 Byte 4: 00000000 Memory Layout: Sign: 0 Exponent: 10000010 Mantissa: 100110000000000000000000
Double	8 Bytes (64 bits)	IEEE 754 Double Precision (1 sign bit, 11 exponent bits, 52 fraction/mantissa bits)	25.6789	Binary: Byte 1: 0 1000000 Byte 2: 00111001 Byte 3: 10101101 Byte 4: 11001100

Data Type	Size	Representation	Example	Memory (Bytes)
		fraction/mantissa bits)		Byte 5: 01100011 Byte 6: 11110001 Byte 7: 01000011 Byte 8: 11110011 Memory Layout: Sign: 0 Exponent: 10000000011 Mantissa: 1001101011011100110001100011111100010100001111110011

2.3.2.2 Primitive Data Objects (Value Domains)

A **data object** is the **collection of all possible values** belonging to a data type. For primitive data types, the data object is simply the set of values that the data type can represent. A data object may be viewed in two ways:

- **Atomic (Primitive):** The value is treated as indivisible.

Example:

675 - is considered a single integer.

GOOD - consists of the characters (String)

- **Composite(Non Primitive):** The value is viewed as a collection of smaller elements.

Example:

675 - consists of the digits 6, 7, 5

G O O D - where each character is an instance of the Character data object.

Data Object	Defined Set	Description
Boolean	{false,true}	true and false are instances of the Boolean data object.
Digit	{0,1,2,3,4,5,6,7,8,9}	0–9 are instances of the Digit data object.
Character	{A,B, ...,Z,a,b, ...,z}	Set of uppercase and lowercase alphabetic characters.
Natural Number	{0,1,2, ... }	Set of non-negative integers.

Data Object	Defined Set	Description
Integer	{..., -2, -1, 0, 1, 2, ...}	Every integer value belongs to the Integer data object.
String	{a, b, ..., aa, ab, abc, ...}	Every string is composed of characters belonging to the Character data object.

iv. Relationship Between Data Type and Data Object

Data Type	Data Object
Defines the kind of data, memory representation, and valid operations.	Defines the collection of all possible values (instances).
Includes operations.	Contains only values.
Example: Integer	{..., -2, -1, 0, 1, 2, ...}
Data Object + Operations + Representation	Set of Values (Domain)

2.3.2.3 Non-Primitive (Composite) Data Types / Non-Primitive Data Structures

i. Definition of Non-Primitive Data Types

Non-primitive data types are programmer-defined or derived from primitive type. It is used to store and organize complex data. It groups multiple values together. This is also called reference types, derived types, or composite types.

ii. Characteristics of Non-Primitive Data Types

- **Memory Storage:** In many programming languages (like Java and C#), non-primitive types are called "reference types." The variable does not store the actual data; instead, it stores a reference (or memory address) pointing to where the data is located on the heap.
- **Default Values:** When initialized without a value, non-primitive types generally default to null (meaning they point to nothing). Primitives usually default to 0 or false.
- **Methods:** Non-primitive types can have methods associated with them. For example, a String (which is an array of characters under the hood) often has built-in methods like `.length()` or `.toUpperCase()`. Primitives cannot have methods.
- **Size:** Unlike primitives, which have a strictly fixed size (e.g., a standard integer is usually 4 bytes), non-primitive types can have variable sizes that grow or shrink dynamically depending on the data they hold.

iii. Classification of Non-Primitive Data Types

- Concrete Data Structures - A concrete data structure specifies the **physical organization and representation of data in memory**. It describes how data elements are stored and related at the implementation level. Example - Arrays , Matrices , Records
- User-Defined Data Types -User-defined data types are created by programmers using the data-type construction mechanisms provided by a programming language. They allow related data items to be grouped and represented according to the requirements of an application. Example – Structure , Union, Enumeration , Class.
- Abstract Data Types (ADTs) - An **Abstract Data Type (ADT)** is a logical specification of a data object and the operations that can be performed on it, independent of its implementation. An ADT specifies **what operations are performed**, but does not specify **how those operations are implemented or how the data is physically stored**. Example – list, stack , queue , tree , graph.

iv. Memory Representation of Non-Primitive Data Types

The memory representation of a non-primitive data type depends on its implementation and the programming language. Unlike primitive data types, which generally have a fixed and predefined representation, non-primitive structures may contain multiple elements and may occupy different amounts of memory.

In languages such as **Java and C#**, many non-primitive entities are handled through **references**, where a variable refers to an object stored elsewhere in memory.

v. Size and Storage

The storage requirement of a non-primitive data type depends on:

- Number of elements
- Type and size of individual elements
- Memory organization
- Static or dynamic allocation
- Implementation of the data structure

For example, an array of 100 integers requires memory proportional to the number of elements, whereas a dynamically allocated linked structure may grow or shrink according to the number of elements.

vi. Operations on Non-Primitive Data Types

Non-primitive data types and data structures support operations appropriate to their organization. Common operations include:

- Traversal
- Searching
- Insertion
- Deletion
- Sorting
- Merging
- Updating
- Accessing

The operations available depend on the particular data structure and its associated ADT

vii. Primitive and Non-Primitive Data Types – Comparison

Primitive Data Types	Non-Primitive Data Types
Store a single value.	Can represent multiple values or complex relationships.
Usually predefined by the language.	May be derived, constructed, or user-defined.
Generally have a fixed representation.	May have fixed or variable memory requirements.
Directly supported by the language and hardware.	Built using primitive types and language mechanisms.
Examples: Integer, Float, Character, Boolean	Examples: Array, Structure, Class, List, Stack, Queue

2.4 Chapter Summary

- Data represents facts, values, or information that can be processed by a computer.
- A data type specifies the set of values that can be stored and the operations that can be performed on those values.
- A data object represents a collection of values or instances belonging to a particular domain.
- Data types provide information about the value domain, representation, storage, and permissible operations.
- Data types are broadly classified into primitive and non-primitive data types.
- Primitive data types are the fundamental data types directly supported by a programming language, such as integer, float, double, character, and Boolean.

- Primitive values are represented internally using bits and bytes in computer memory.
- Different primitive data types use different memory representation techniques, such as two's complement, ASCII/Unicode, and IEEE 754.
- Non-primitive data types represent collections of values or more complex relationships among data elements.
- Non-primitive data types include concrete data structures, user-defined data types, and abstract data types (ADTs).
- Arrays are important concrete data structures that organize multiple elements, generally of the same type, in a structured memory layout.
- User-defined data types, such as structures, unions, enumerations, and classes, allow programmers to create data representations suited to specific applications.
- An Abstract Data Type (ADT) specifies the data and the operations that can be performed on it without specifying its implementation.
- The concepts of data type, data object, memory representation, operations, and data structures provide the foundation for studying data structures and algorithms.
- A clear understanding of these concepts helps in selecting and designing appropriate and efficient data structures for solving computational problems.

3 One Dimensional and Multi-Dimensional Arrays

An **array** is a non-primitive data structure used to store a collection of elements of the **same data type**. The elements of an array are stored in a well-defined order and can be accessed using an **index or subscript**. In a conventional array implementation, the elements occupy contiguous memory locations, which enables efficient direct access.

Arrays can be classified according to their number of dimensions. The two important forms are **one-dimensional arrays** and **multi-dimensional arrays**.

3.1 One-Dimensional (1D) Arrays

A **one-dimensional (1D) array** is a linear collection of elements arranged sequentially in memory. Each element is identified using a single index.

3.1.1 Characteristics of One-Dimensional Arrays

- Stores multiple elements of the same data type.
- Elements are arranged sequentially.
- Each element is identified by a single index.
- Supports direct or random access.
- In a conventional array, elements occupy contiguous memory locations.
- The size is generally fixed when the array is allocated.
- Address calculation allows an element to be accessed in **O(1)** time.

3.1.2 Memory Representation of a One-Dimensional Array

Consider the following array:

A = [10, 20, 30, 40, 50]

The logical representation is:

Index	0	1	2	3	4
	▼	▼	▼	▼	▼
A	10	20	30	40	50
Address	1000	1004	1008	1012	1016

If each integer occupies **4 bytes**, the elements are stored at consecutive memory locations. Let, **BA** = Base Address of the array , **w** = Size of each element in bytes , **i** = Index of the required element , For a **0-based array**, the address of A[i] is: $Address(A[i]) = BA + (i \times w)$

Example

Suppose:

Base Address BA = 1000

Size of each integer $w = 4$ bytes

Required element = $A[3]$, $i=3$

Then: $\text{Address}(A[3]) = 1000 + (3 \times 4) = 1000 + 12 = 1012$

Therefore, the fourth element is stored at memory address **1012**.

3.2 Multi-Dimensional Arrays

A **multi-dimensional array** is an array that uses more than one index to identify an element. It is commonly used to represent tables, matrices, grids, and higher-dimensional data.

A two-dimensional array uses **rows and columns**, while a three-dimensional array uses three indices.

For example, a two-dimensional array can be represented as:

The element $A[1][2]$ represents the element in **row 1 and column 2**, which is 6.

Although a multi-dimensional array is logically represented as rows and columns, the physical memory of a computer is organized as a linear sequence of memory locations. Therefore, the compiler or programming language maps the multi-dimensional array into a linear memory sequence.

The two commonly used storage mappings are:

Row-Major Order

Column-Major Order

3.2.1 Row-Major Order

In **row-major order**, the elements are stored **row by row**. All elements of the first row are stored first, followed by all elements of the second row, and so on.

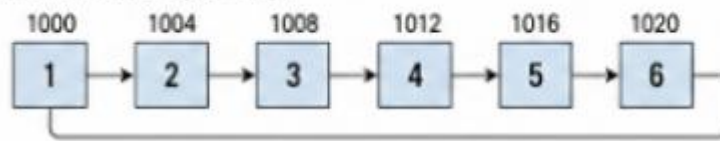
Consider the following 2×3 array:

Row 0	1	2	3
Row 1	4	5	6
	Column 0	Column 1	Column 2

THEORY:
Easy for humans to visualize. Access data points using (Row, Column).

A. ROW-MAJOR ORDER

(Rows stored sequentially)



Thus, the physical memory sequence is:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$

Row-Major Address Calculation

For a two-dimensional array $A[i][j]$ having **N columns**:

where: **BA** = Base Address , **i** = Row index , **j** = Column index , **N** = Number of columns

w = Size of each element in bytes

ROW-MAJOR ADDRESS FORMULA:

$$\text{Address}(\text{row}, \text{col}) = \text{Base} + ((\text{row} * \text{NumCols} + \text{col}) * \text{Size})$$

Example

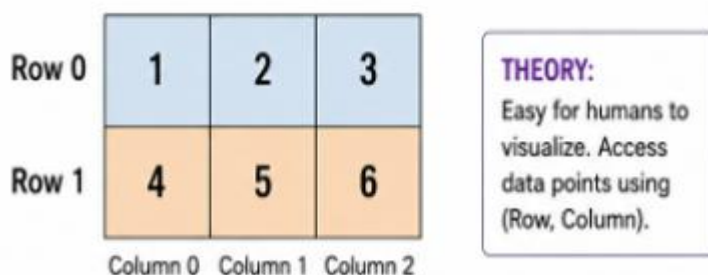
For the above 2×3 array, assume:

Base Address BA= 1000 , Element size $w = 4$ bytes , Number of columns = 3, row=1, Col=2, $A[1][2]$ is required = $1000 + ((1*3+2)*4) = 1020$

3.2.2 Column-Major Order

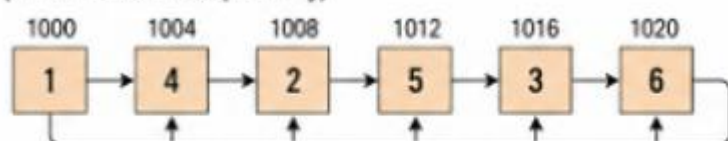
In **column-major order**, the elements are stored **column by column**. All elements of the first column are stored first, followed by all elements of the second column, and so on.

For the same 2×3 array:



B. COLUMN-MAJOR ORDER

(Columns stored sequentially)



Thus, the memory sequence is: $1 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 6$

Column-Major Address Calculation

For a two-dimensional array $A[i][j]$ having **M** rows:

where: **BA** = Base Address , **i** = Row index , **j** = Column index , **M**= Number of rows
w = Size of each element in bytes

COLUMN-MAJOR ADDRESS FORMULA:

$$\text{Address}(\text{row}, \text{col}) = \text{Base} + ((\text{col} * \text{NumRows} + \text{row}) * \text{Size})$$

Example

For the above 2×3 array, assume:

Base Address BA= 1000 , Element size $w = 4$ bytes , Number of rows = 2, row=1, Col=2,
 $A[1][2]$ is required = $1000 + ((2*2+1)*4) = 1016$

3.2.3 Higher-Dimensional Arrays

Arrays can have more than two dimensions. A **three-dimensional (3D) array** uses three indices and can be used to represent data such as a collection of matrices.

For example: $A[i][j][k]$

where:

i represents the first dimension,

j represents the second dimension, and

k represents the third dimension.

Similarly, arrays can be extended to four or more dimensions depending on the requirements of an application.

3.2.4 Applications of Arrays

Arrays are widely used in:

- Storing collections of homogeneous data.
- Representing matrices and tables.
- Implementing stacks and queues.
- Searching and sorting algorithms.
- Representing strings and character sequences.
- Storing numerical and scientific data.
- Implementing other data structures and algorithms.

3.2.5 *Advantages and Disadvantage of Arrays*

3.2.5.1 Advantages

- Direct and fast access to elements.
- Efficient use of contiguous memory.
- Simple and easy to implement.
- Suitable for sequential processing.
- Provides $O(1)$ indexed access.

3.2.5.2 Disadvantage

- Conventional arrays have a fixed size.
- Insertion and deletion in the middle can be costly.
- Requires contiguous memory allocation.
- May result in unused memory when the allocated size is larger than required.
- Stores elements of the same data type in a conventional homogeneous array.

3.3 Summary

- A **one-dimensional array** organizes elements in a linear sequence.
- Each element in a one-dimensional array is accessed using a **single index**.
- Contiguous memory allocation enables **direct address calculation**.
- Array elements can be accessed in **$O(1)$ constant time**.
- A **multi-dimensional array** uses two or more indices to represent structured data.
- Multi-dimensional arrays are commonly used to represent **matrices, tables, grids, and higher-dimensional data**.
- Since computer memory is logically a **linear sequence of memory locations**, multi-dimensional arrays are mapped into linear memory.
- **Row-major order** stores array elements **row by row**.
- **Column-major order** stores array elements **column by column**.
- Address calculation determines the memory location of an array element using the **base address, index, dimensions, and element size**.
- The study of one-dimensional and multi-dimensional arrays provides a foundation for understanding **memory organization, address calculation, and complex data structures**.

4 Abstract Data Types

4.1 Definition of Abstract Data Type

An **Abstract Data Type (ADT)** is a logical specification of a data type that defines the **data objects and the operations** that can be performed on them, while hiding the details of their internal representation from the user or client.

An ADT describes **what** operations are available and **what they do**, but does not specify **how** those operations are implemented.

For example, a **Stack ADT** specifies operations such as `push`, `pop`, and `peek`. The stack may be implemented using an array or a linked list, but the user of the stack need not know the underlying implementation.

4.2 Components of an ADT

An Abstract Data Type consists of two major components.

4.2.1 Data Objects

The data objects define the **set of values or elements** that belong to the ADT. For example, a Stack may contain a collection of elements arranged according to the **Last-In-First-Out (LIFO)** principle. To make it clear think of an Abstract Data Type (ADT) as a specialized storage box with strict rules, and the Data Objects as the items you place inside that box. Let's break it down into two parts: the **What** and the **How**.

4.2.1.1 The "What" (The Data Objects)

Data objects are the actual pieces of information you are working with. Depending on your program, a data object could be:

- An integer (like the number 42)
- A string of text (like "Hello World")
- A complex record (like a Student profile with a name, ID, and grades)

4.2.1.2 The "How" (The ADT's Rules)

The ADT defines how these data objects are organized and accessed. The data objects don't just float around randomly; they have to follow the rules of the specific ADT they belong to. The ADT gives the information how these data objects are related. For example, in a Stack, the **Data Objects** are the items being stacked, and **LIFO** is the rule dictating how you can interact with them.

4.2.2 Operations

Operations define the valid actions that can be performed on the data objects.

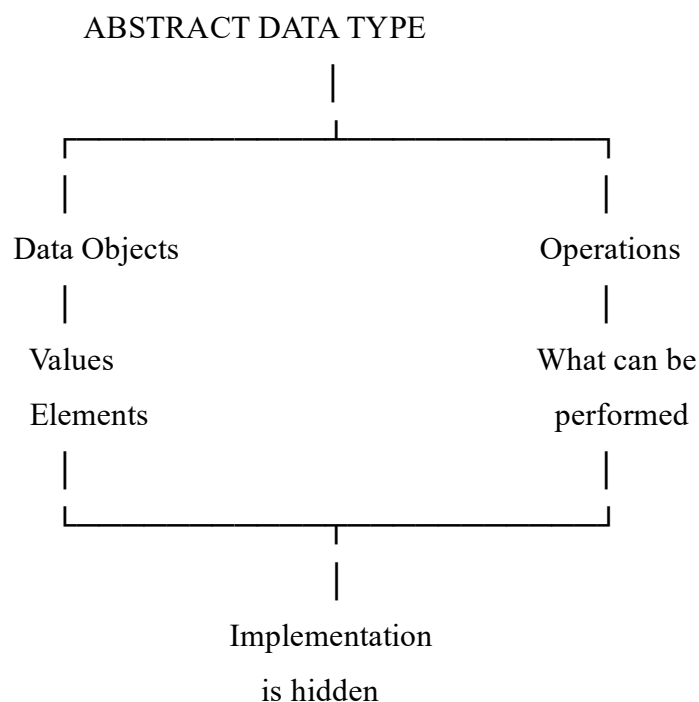
For example, common Stack operations include:

- **Push** – inserts an element into the stack.
- **Pop** – removes the top element from the stack.
- **Peek/Top** – returns the top element without removing it.
- **isEmpty** – determines whether the stack is empty.

4.2.3 Abstraction and Encapsulation

An ADT provides **abstraction** by allowing users to work with a data type through its defined operations without knowing its implementation details.

It also supports **encapsulation** by combining the data and the operations that manipulate the data while restricting direct access to the internal representation.



4.2.4 Interface and Implementation

An ADT can be viewed through two distinct aspects:

4.2.4.1 Interface

The **interface** specifies the operations available to the user and the behavior expected from those operations. The client interacts with the ADT through this interface.

4.2.4.2 Implementation

The **implementation** specifies how the data is represented and how the operations are internally performed. The implementation may use different data structures without changing the interface provided to the client.

4.2.4.3 ADT and Data Representation

When **using an ADT**, the user focuses on:

- Available operations
- Input and output of operations
- Behavior of the operations
- Rules and constraints of the ADT

The user does **not** need to know how the data is represented internally. When **implementing an ADT**, the programmer focuses on:

- Selecting an appropriate data representation
- Storing the data
- Designing the operations
- Maintaining the properties of the ADT
- Ensuring efficient implementation

4.2.4.4 Example: Stack as an ADT

A **Stack** is an example of an Abstract Data Type.

- **Data Object**

A collection of elements organized according to the **Last-In-First-Out (LIFO)** principle.

- **Operations**

Operation	Description
push(x)	Inserts element x at the top
pop()	Removes and returns the top element
peek()	Returns the top element without removing it
isEmpty()	Checks whether the stack is empty

- The Stack ADT can be implemented using:

An **array**

A **linked list**

The user of the Stack ADT does not need to know which implementation is being used.

4.2.4.5 Examples of Abstract Data Types

Common ADTs include:

- **List ADT**
- **Stack ADT**
- **Queue ADT**
- **Deque ADT**
- **Priority Queue ADT**
- **Set ADT**
- **Map ADT**
- **Dictionary ADT**

More complex structures such as **trees and graphs** can also be specified and used through abstract interfaces.

4.2.4.6 ADT and Data Structure

An important distinction is that an ADT describes the logical behavior, whereas a data structure provides the implementation.

Abstract Data Type	Data Structure
Logical specification	Physical implementation
Defines what operations are available	Defines how operations are implemented
Hides representation	Provides representation
Independent of implementation	Dependent on implementation
Example: Stack ADT	Array-based Stack or Linked-List Stack

4.2.4.7 Importance of Abstract Data Types

Abstract Data Types are important in software and program design because they:

- Support abstraction.
- Promote encapsulation.
- Hide implementation details.
- Separate interface from implementation.
- Improve program modularity.
- Make programs easier to maintain and modify.
- Allow the same ADT to have multiple implementations.
- Promote code reuse.
- Make complex programs easier to design and understand.
- Allow implementation details to change without affecting the client code.

4.2.4.8 ADTs in Object-Oriented Programming

In object-oriented programming, an ADT can be implemented using a **class**. The class combines data and the methods that operate on that data while controlling access to its internal representation.

For example, a C++ class can implement a Stack ADT by maintaining an internal array and providing methods such as `push()`, `pop()`, and `peek()`.

The client interacts with the **public methods**, while the internal data representation can be kept private.

4.3 Summary

- An **Abstract Data Type** defines a data object and the operations that can be performed on it without exposing its internal representation.
- It emphasizes the **behavior and interface** of a data type rather than its implementation.
- The distinction between an ADT and a data structure is fundamental: **an ADT specifies what a data type does, whereas a data structure specifies how it is implemented.**
- This separation enables different data structures to implement the same ADT and provides flexibility, modularity, and maintainability in software design.
- $\text{ADT} = \text{Data Objects} + \text{Operations} + \text{Hidden Representation}$. This makes **Abstract Data Types** an important bridge between the fundamental concepts of **data types and data objects** and the practical implementation of **data structures**.

5 Linear Lists – List ADT

5.1 Introduction to Linear Lists

A **linear list** is a finite sequence of elements in which the elements are arranged in a definite linear order. Each element has a unique position in the sequence, except that the same value may occur more than once.

A **linear list** can be represented as: $L = (a_1, a_2, a_3, \dots, a_n)$

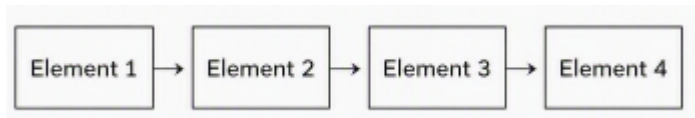
$a_1 \rightarrow$ first element

$a_n \rightarrow$ last element

$n \rightarrow$ number of elements in the list

The **sequential arrangement of elements** is the fundamental property of a linear list.

For example:



The elements are arranged from the **first element to the last element**, establishing a linear relationship among them.

5.2 Types of Linear Lists

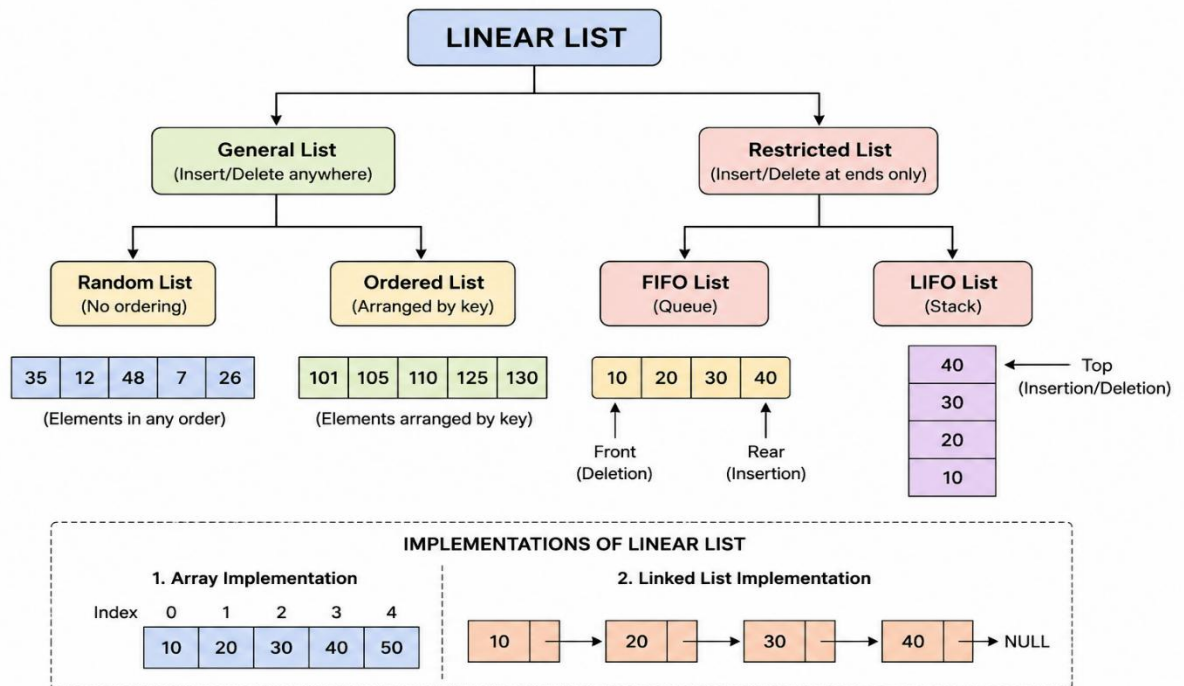
Linear lists are classified into two main categories:

5.2.1 General Lists

- *Data can be inserted or deleted anywhere without restrictions.*
- Unordered List: No specific sequence of data.
- Ordered List: Data arranged according to a *key* (e.g., employee number in employee record, Register number in student record).

5.2.2 Restricted Lists

- *Data can only be added or removed at the ends of the structure.*
- FIFO (Queue): First-In-First-Out order.
- LIFO (Stack): Last-In-First-Out order



5.3 Linear List as an Abstract Data Type

5.3.1 Linear List as an Abstract Data Type

A Linear List ADT defines a collection of elements and the operations that can be performed on those elements. It specifies the logical behavior of the list without requiring a particular implementation.

The list may be implemented using an array, linked list, or other suitable data structure.

Thus, the Linear List ADT specifies what operations are performed, while the implementation specifies how those operations are performed. The same Linear List ADT can therefore have different implementations.

5.3.2 Characteristics of a Linear List

The important characteristics of a linear list are:

- Elements are arranged in a **linear sequence**.
- Every element has a definite **position** in the list.
- The list has a **first element** and, if non-empty, a **last element**.
- Each element, except the first, has a **predecessor**.
- Each element, except the last, has a **successor**.
- Elements can generally be **inserted, deleted, searched, accessed, or updated**.
- The number of elements in the list may vary depending on its implementation.

5.3.3 *Operations of Linear List ADT*

A Linear List ADT generally supports a collection of operations for creating and manipulating the list.

Create - Creates an empty list or initializes a list with a given set of elements.

Insert - Adds a new element at a specified position in the list.

Delete - Removes an element from a specified position.

Search - Determines whether a specified element exists in the list and identifies its position if required.

Retrieve or Access - Obtains the element stored at a specified position.

Update - Changes the value of an existing element.

Traverse - Visits or processes each element of the list sequentially.

IsEmpty - Determines whether the list contains any elements.

Length or Size - Returns the number of elements currently present in the list.

5.3.4 *Linear List ADT and Its Implementations*

The Linear List ADT can be implemented using different concrete data structures.

5.3.4.1 Array-Based Implementation

In this implementation, statically allocated array is used. Because static arrays have a fixed size, there is a need to maintain a separate variable to track the *actual* number of elements currently stored in the list.

Key Concept: Capacity vs. Size

Capacity: The maximum number of items the array can hold (e.g., 100).

Size: The current number of valid items actively stored in the list.

i. The Class Structure

To build Array List, bundle the array and the size tracker into a single class. When a new list is created, it is empty. Set the size counter to zero, meaning the first inserted element will automatically go to index 0.

```

class ArrayList {
private:
    int arr[100]; // The contiguous block of memory (Capacity = 100)
    int size;      // The counter tracking active elements

public:
    // Constructor initializes an empty list
    ArrayList() {
        size = 0;
    }
}

```

ii. Insertion Operations

Adding elements to an array requires to handle two scenarios: adding to the very end, and inserting into the middle.

Pre-condition for all insertions: check for **Overflow**. If `size == 100`, the array is full, and cannot insert new data. Overflow Condition - If the array has a capacity of 100 and `size == 100` the array is full. No additional element can be inserted. This condition is called **overflow**. **Overflow occurs when an insertion is attempted in a list whose underlying array has reached its maximum capacity.**

- Appending (insertAtlast)

Adding an element to the end of the list is fast and efficient. The algorithm places the new value at the first available empty slot (`arr[size]`) and increments the size counter.

```

void insertAtlast(int value) {
    // Overflow check
    if (size == 100) {
        cout << "List is full!" << endl;
        return;
    }

    // Insert and increment size
    arr[size++] = value;
}

```

- Inserting at a Specific Position (insertAt)

Inserting an element into the middle of the list requires making room for it. Starting from the *end* of the list, move each element one position to the right until the target position is reached, vacating a slot for the new value.

Steps for Insertion at a Position

1. Check for overflow.
2. Validate the position.
3. Shift the existing elements one position to the right, starting from the last element.
4. Insert the new value at the specified position.
5. Increment size.
6. The elements must be shifted from right to left so that existing values are not overwritten.

```
void insertAt(int pos, int value) {  
    // Validate capacity and index boundaries  
    if (size == 100 || pos < 0 || pos > size) {  
        cout << "Invalid position!" << endl;  
        return;  
    }  
  
    // Shift elements to the right to create a gap  
    for (int i = size; i > pos; i--) {  
        arr[i] = arr[i - 1];  
    }  
  
    // Insert the new value and update size  
    arr[pos] = value;  
    size++;  
}
```

iii. Deletion Operation

Deletion is the operation of removing an element from the list. When an element is deleted from the middle or beginning of an array-based list, a gap would be created if the remaining elements were not moved. To maintain the logical order of the list, the elements following the deleted element must be shifted one position to the left.

Underflow Condition - Before performing a deletion, the list must be checked to determine whether it contains any elements.

If `size == 0` - the list is empty, and no element can be deleted. This condition is called **underflow**. **Underflow occurs when a deletion operation is attempted on an empty list.**

- Removing an Element (`deleteAt`)

The operation `deleteAt(pos)` removes the element at the specified position.

Steps

1. Check whether the list is empty.

2. Validate the position.
3. Shift all elements following the deleted position one position to the left.
4. Decrement size.
5. The shifting operation is performed using:
6. `arr[i] = arr[i + 1];`

```
void deleteAt(int pos) {
    // Validate index against active size
    if (pos < 0 || pos >= size) {
        cout << "Invalid position!" << endl;
        return;
    }

    // Shift elements to the left to close the gap
    for (int i = pos; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }

    // Decrement size counter
    size--;
}
```

iv. Searching

Linear Search algorithm is used to find specific values. It loops through the active array, returning the target's index if found, or -1 if the item does not exist.

```
int search(int value) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == value)
            return i;
    }
    return -1; // Value not found
}
```

v. Traversal (display)

Traversal is the process of visiting or processing each element of the list exactly once.

In an array-based list, traversal is performed from index 0 through `size - 1`.

For example:

```
for (int i = 0; i < size; i++) {
    cout << arr[i] << " ";}
```

If: `size = 4` only the following positions are traversed: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

The remaining positions of the array are not considered part of the logical list.

Empty List - If: `size == 0` the list contains no valid elements. The program may display:

List is empty.

5.3.4.2 Advantages of Array-Based List

Fast Random Access – Direct index access in $O(1)$.

Simple Implementation – Straightforward array structure.

Efficient Memory Organization – Contiguous storage improves locality.

Efficient Appending – Adding at end is $O(1)$ if capacity available.

Low Overhead – No extra pointers as in linked lists.

5.3.4.3 Limitations of Array-Based List

Fixed Capacity – Cannot grow beyond predefined size.

Costly Insertions – Shifting required, worst case $O(n)$.

Costly Deletions – Shifting required, worst case $O(n)$.

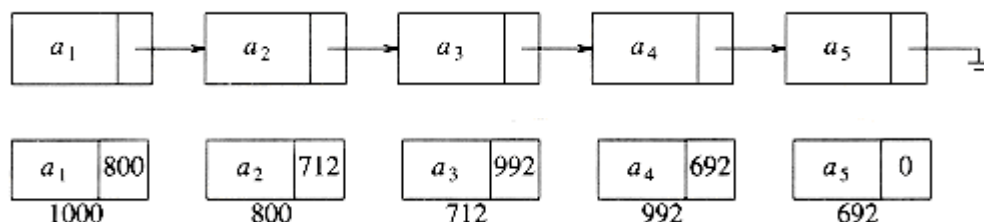
Unused Space – Large capacity may leave many empty slots.

Example: Capacity = 100, Size = 10 $\rightarrow$ 90 unused positions.

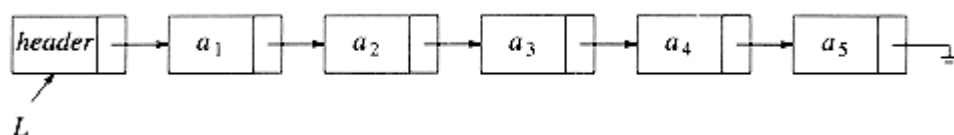
5.3.4.4 Linked list -Based Implementation

In an array-based list, elements are stored in contiguous memory locations. In contrast, a linked list stores elements in separate memory locations and connects them using links or pointers. A linked list consists of nodes, each containing **Data** and a **Link** (pointer to the next node). In a singly linked list, each node points to its successor, and the last node points to nullptr.

1 List without header



2 List with header

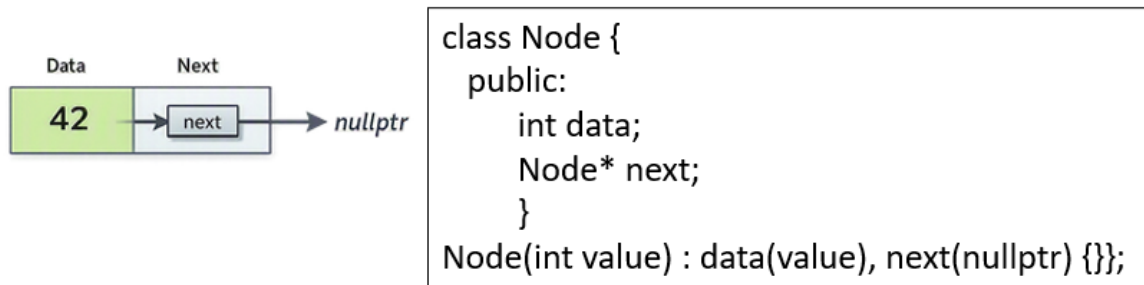


i. Nodes in a Linked List

The elements of a linked list are traditionally called **nodes**.

A node in a singly linked list contains two fields:

- **Data field:** Stores the actual value or information.
- **Next field:** Stores the address of the next node.
- The nodes do not have to occupy consecutive memory locations. Their next fields establish the logical relationship between them.



- **Self-Referential Structures** - A structure that contains a pointer to another instance of the same structure type is called a **self-referential structure**.
- **The Head Pointer** - A linked list requires a pointer to identify its first node. This pointer is generally called **head**.
- `Node* head;` - The head pointer stores the address of the first node in the list.
- **Empty Linked List** - An empty linked list does not contain any nodes. Therefore, the head pointer is set to: `head = nullptr;`

ii. Linked List as an Implementation of the List ADT

The List ADT specifies what operations a list should support, while the linked list specifies how those operations are implemented.

Typical List ADT operations include:

- Insert an element
- Delete an element
- Search for an element
- Access an element
- Traverse the list
- Check whether the list is empty

A singly linked list can implement these operations using nodes and links.

The important distinction is:

The List ADT describes the behavior of the list, whereas the linked list provides a concrete representation for implementing that behavior.

iii. Dynamic Memory Allocation

Unlike a statically allocated array, linked-list nodes are generally created dynamically as they are required.

In C++, dynamic memory can be allocated using the `new` operator.

```
Node* newNode = new Node;
```

A node can then be initialized as: `Node* newNode = new Node{value, nullptr};`

iv.

• 11. Summary

- A **data type** specifies the kind of values that can be stored and the operations that can be performed on them.
- Data types ensure efficient memory usage, type safety, and reliable program execution.
- **Primitive data types** store individual values and are directly supported by the programming language.
- **Non-primitive data types** combine multiple values and form the basis for data structures such as arrays, linked lists, stacks, queues, trees, and graphs.